IRC Socket - Source Code

Group 4

23 October 2026

Contents

FILE	LINES	PAGE
bot/__init__.py	1	3
bot/__main__.py	4	3
bot/cli.py	31	3
bot/client.py	329	4
bot/commands.py	98	9
bot/data/facts.txt	5	11
bot/queries.py	204	11
bot/state.py	107	15
common/__init__.py	1	17
common/irc_message.py	133	17
common/numerics.py	27	19
common/protocol.py	44	20
common/stream.py	41	21
server/__init__.py	1	22
server/__main__.py	4	22
server/cli.py	30	22
server/client_session.py	197	23
server/errors.py	5	26
server/handlers.py	394	26
server/interfaces.py	48	33
server/server.py	231	34
server/state.py	195	38

bot/__init__.py

```
"""IRC command bot."""
```

bot/__main__.py

```
from .cli import main

if __name__ == "__main__":
    main()
```

bot/cli.py

```
"""Command-line configuration for the bot."""

import argparse
import logging

from .client import BotClient


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="IPv6 IRC bot")
    parser.add_argument("--host", default="fc00:1337::17")
    parser.add_argument("--port", type=int, default=6667)
    parser.add_argument("--name", default="SuperBot", dest="nickname")
    parser.add_argument("--channel", default="#hello")
    return parser


def main(argv: list[str] | None = None) -> None:
    args = build_parser().parse_args(argv)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    try:
        client = BotClient(args.host, args.port, args.nickname, args.channel)
    except ValueError as exc:
        build_parser().error(str(exc))
    try:
        client.run()
    except KeyboardInterrupt:
        client.stop()
```

bot/client.py

```
"""IPv6 IRC bot client with registration, keep-alive and reconnect support."""

from __future__ import annotations

import logging
from collections import deque
from collections.abc import Callable
from functools import partial
import socket
import threading
import time

from common.irc_message import IRCMessage, message
from common.numerics import (
    RPL_ENDOFNAMES,
    RPL_NAMREPLY,
    RPL_WELCOME,
)
from common.protocol import irc_casefold, is_channel, valid_nickname
from common.stream import IRCStreamDecoder, IRCStreamError

from .commands import BotCommandProcessor, CommandAction
from .queries import BotQueries
from .state import BotState

logger = logging.getLogger(__name__)


class BotClient:
    REGISTRATION_TIMEOUT = 30.0
    IDLE_TIMEOUT = 30.0
    PONG_TIMEOUT = 30.0
    NAMES_TIMEOUT = 30.0
    MAX_PENDING_COMMANDS = 100
    TEXT_CHUNK_BYTES = 256

    def __init__(
        self,
        host: str = "fc00:1337::17",
        port: int = 6667,
        nickname: str = "SuperBot",
        channel: str = "#hello",
        reconnect_delay: float = 3.0,
    ) -> None:
        if not 1 <= port <= 65535:
            raise ValueError("port must be between 1 and 65535")
        self.host, self.port, self.nickname, self.channel = (
            host,
            port,
            nickname if valid_nickname(nickname) else "SuperBot",
            channel,
        )
        if not is_channel(self.channel):
            raise ValueError("channel must be a valid IRC channel name")
        self.reconnect_delay = reconnect_delay
        self.state = BotState()
        self.commands = BotCommandProcessor(self.nickname)
        self.queries = BotQueries()
        self._action_handlers: dict[CommandAction, Callable[[IRCMessage], None]] = {
            CommandAction.WAIT_FOR_NAMES: self._defer_command,
```

bot/client.py (continued)

```
            CommandAction.REQUEST_WHO: self._request_who,
            CommandAction.REQUEST_CHANNELS: partial(self._request_query, "LIST"),
            CommandAction.REQUEST_NETWORK: partial(self._request_query, "LUSERS"),
        }
        self.socket: socket.socket | None = None
        self.send_lock = threading.Lock()
        self.stopping = threading.Event()
        self._registered = False
        self._server_name = "s" * 63
        self._nick_attempt = 0
        self._names_started: float | None = None
        self._pending_commands: deque[IRCMessage] = deque()

    def run(self) -> None:
        while not self.stopping.is_set():
            try:
                self.run_once()
            except (ConnectionError, OSError, IRCStreamError) as exc:
                if not self.stopping.is_set():
                    logger.warning(
                        "Bot connection to [%s]:%s ended: %s", self.host, self.port, exc
                    )
            except Exception:
                logger.exception("Unexpected bot protocol error")
                raise
            finally:
                self.close()
            # A bounded delay avoids a tight reconnect loop when the server is
            # unavailable, while still allowing the bot to recover on its own.
            if not self.stopping.wait(self.reconnect_delay):
                continue

    def run_once(self) -> None:
        # Discard stale membership after reconnecting. JOIN events and the new
        # NAMES snapshot rebuild it; commands may arrive before NAMES completes.
        self.state.clear()
        self.queries.reset()
        self._names_started = None
        self._pending_commands.clear()
        self._registered = False
        self._nick_attempt = 0
        sock = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
        self.socket = sock
        if self.stopping.is_set():
            return
        sock.settimeout(1.0)
        sock.connect((self.host, self.port))
        started = last_received = time.monotonic()
        ping_sent: float | None = None
        ping_token: str | None = None
        decoder = IRCStreamDecoder()
        self.send(message("NICK", self.nickname))
        self.send(message("USER", self.nickname, "0", "*", self.nickname))
        while not self.stopping.is_set():
            self._check_timeouts(started, ping_sent)
            now = time.monotonic()
            if (
                self._registered
                and ping_sent is None
                and now - last_received >= self.IDLE_TIMEOUT
```

bot/client.py (continued)

```
            ):
                ping_token = f"bot-{now:.9f}"
                ping_sent = now
                self.send(message("PING", ping_token))
            try:
                data = sock.recv(4096)
            except socket.timeout:
                continue
            if not data:
                raise ConnectionError("server closed connection")
            self._check_timeouts(started, ping_sent)
            for incoming in decoder.feed(data):
                last_received = time.monotonic()
                if (
                    incoming.command == "PONG"
                    and incoming.params
                    and incoming.params[-1] == ping_token
                ):
                    ping_sent = None
                    ping_token = None
                self.handle(incoming)

    def _begin_names(self, channel: str) -> None:
        if not self.state.has_pending_names(channel):
            self.state.begin_names(channel)
            if irc_casefold(channel) == irc_casefold(self.channel):
                self._names_started = time.monotonic()

    def _check_timeouts(self, started: float, ping_sent: float | None) -> None:
        now = time.monotonic()
        if not self._registered and now - started >= self.REGISTRATION_TIMEOUT:
            raise TimeoutError("IRC registration timed out")
        if ping_sent is not None and now - ping_sent >= self.PONG_TIMEOUT:
            raise TimeoutError("server did not answer PING with matching PONG")
        if (
            self._names_started is not None
            and now - self._names_started >= self.NAMES_TIMEOUT
        ):
            self._names_started = None
            self._pending_commands.clear()
            raise TimeoutError("NAMES snapshot timed out")
        self.queries.check_timeouts(now)

    def send(self, msg: IRCMessage) -> None:
        with self.send_lock:
            sock = self.socket
            if sock is None:
                raise ConnectionError("bot is not connected")
            sock.sendall(msg.to_bounded_bytes())

    def handle(self, incoming: IRCMessage) -> None:
        command = incoming.command
        outgoing = self.queries.handle(incoming, self.nickname)
        if outgoing is not None:
            self._send_query_messages(outgoing)
            return
        if command in {"432", "433"} and not self._registered:
            self._nick_attempt += 1
            if self._nick_attempt > 10:
                raise ConnectionError("nickname registration rejected repeatedly")
```

bot/client.py (continued)

```
181             base = self.nickname if valid_nickname(self.nickname) else "SuperBot"
182             suffix = "_" + str(self._nick_attempt)
183             self.nickname = base[: 30 - len(suffix)] + suffix
184             self.commands.nickname = self.nickname
185             self.send(message("NICK", self.nickname))
186             return
187         if command == "PING":
188             token = incoming.params[-1] if incoming.params else ""
189             self.send(message("PONG", token))
190             return
191         if command == RPL_WELCOME:
192             self._registered = True
193             if incoming.prefix:
194                 self._server_name = incoming.prefix
195             if incoming.params:
196                 self.nickname = incoming.params[0]
197                 self.commands.nickname = self.nickname
198             # Registration completes only after the server sends 001; joining
199             # earlier can be rejected by a compliant server.
200             self._begin_names(self.channel)
201             self.send(message("JOIN", self.channel))
202             return
203         if command == "PRIVMSG":
204             self._handle_privmsg(incoming)
205             return
206         self._handle_channel_event(incoming)
207 
208     def _handle_channel_event(self, incoming: IRCMessage) -> None:
209         command = incoming.command
210         if command == RPL_NAMREPLY and len(incoming.params) >= 4:
211             # A large channel may be split across several 353 replies. Keep
212             # collecting names until 366 marks the end of this snapshot.
213             channel = incoming.params[2]
214             self._begin_names(channel)
215             names = incoming.params[3].lstrip(":").split()
216             self.state.add_names(
217                 channel,
218                 [name.lstrip("@+%~&") for name in names],
219             )
220             return
221         if command == RPL_ENDOFNAMES and len(incoming.params) >= 2:
222             channel = incoming.params[1]
223             self.state.finish_names(channel)
224             if irc_casefold(channel) == irc_casefold(self.channel):
225                 self._names_started = None
226                 pending = list(self._pending_commands)
227                 self._pending_commands.clear()
228                 for request in pending:
229                     self.handle(request)
230             return
231         if command == "JOIN" and incoming.prefix and incoming.params:
232             nickname = incoming.prefix.split("!", 1)[0]
233             channel = incoming.params[0]
234             if irc_casefold(nickname) == irc_casefold(self.nickname):
235                 self._begin_names(channel)
236             self.state.joined(channel, nickname)
237             return
238         if command == "PART" and incoming.prefix and incoming.params:
239             self.state.left(incoming.params[0], incoming.prefix.split("!", 1)[0])
240             return
```

bot/client.py (continued)

```
241         if command == "QUIT" and incoming.prefix:
242             self.state.removed(incoming.prefix.split("!", 1)[0])
243             return
244         if command == "NICK" and incoming.prefix and incoming.params:
245             old = incoming.prefix.split("!", 1)[0]
246             new = incoming.params[0]
247             self.state.renamed(old, new)
248             if irc_casefold(old) == irc_casefold(self.nickname):
249                 self.nickname = new
250                 self.commands.nickname = new
251             return
252 
253     def _handle_privmsg(self, incoming: IRCMessage) -> None:
254         if len(incoming.params) < 2 or not incoming.prefix:
255             return
256         sender = incoming.prefix.split("!", 1)[0]
257         target, text = incoming.params[0], incoming.params[1]
258         if irc_casefold(target) == irc_casefold(self.nickname):
259             self._send_text(sender, self.commands.private_reply())
260             return
261         if irc_casefold(target) != irc_casefold(self.channel):
262             return
263         response = self.commands.channel_command(
264             text,
265             sender,
266             self.state.members(target),
267             members_complete=not self.state.has_pending_names(target),
268         )
269         if isinstance(response, CommandAction):
270             self._action_handlers[response](incoming)
271         elif response:
272             self.send(message("PRIVMSG", target, response))
273 
274     def _defer_command(self, incoming: IRCMessage) -> None:
275         if len(self._pending_commands) < self.MAX_PENDING_COMMANDS:
276             self._pending_commands.append(incoming)
277             return
278         self.send(
279             message(
280                 "PRIVMSG",
281                 incoming.params[0],
282                 "Channel roster is loading and the command queue is full; please retry.",
283             )
284         )
285 
286     def _request_who(self, incoming: IRCMessage) -> None:
287         channel = incoming.params[0]
288         self._send_query_messages(
289             self.queries.start_who(
290                 channel, self.state.members(channel), self.nickname,
291                 self._server_name, time.monotonic(),
292             )
293         )
294 
295     def _request_query(self, command: str, incoming: IRCMessage) -> None:
296         self._send_query_messages(
297             self.queries.start(command, incoming.params[0], time.monotonic())
298         )
299 
300     def _send_query_messages(self, outgoing: list[IRCMessage]) -> None:
```

bot/client.py (continued)

```
301         for reply in outgoing:
302             if reply.command == "PRIVMSG":
303                 self._send_text(reply.params[0], reply.params[1])
304             else:
305                 self.send(reply)
306 
307     def _send_text(self, target: str, text: str) -> None:
308         payload = text.encode("utf-8")
309         offset = 0
310         # Reserve space for nickname, username and IPv6 host fields added by
311         # the server. Split only at complete code points and retain all text.
312         while offset < len(payload):
313             text = payload[offset : offset + self.TEXT_CHUNK_BYTES].decode(
314                 "utf-8", errors="ignore"
315             )
316             self.send(message("PRIVMSG", target, text))
317             offset += len(text.encode("utf-8"))
318 
319     def close(self) -> None:
320         sock, self.socket = self.socket, None
321         if sock:
322             try:
323                 sock.close()
324             except OSError as exc:
325                 logger.debug("Bot socket close failed: %s", exc)
326 
327     def stop(self) -> None:
328         self.stopping.set()
329         self.close()
```

bot/commands.py

```
  1 """Pure Bot command decisions, kept independent from sockets."""
  2 
  3 from __future__ import annotations
  4 
  5 import random
  6 from datetime import datetime
  7 from enum import Enum, auto
  8 from pathlib import Path
  9 from common.protocol import irc_casefold
 10 
 11 
 12 class CommandAction(Enum):
 13     REQUEST_WHO = auto()
 14     REQUEST_CHANNELS = auto()
 15     REQUEST_NETWORK = auto()
 16     WAIT_FOR_NAMES = auto()
 17 
 18 
 19 class BotCommandProcessor:
 20     def __init__(
 21         self, nickname: str, facts_path: Path | None = None, rng: random.Random | None = None
 22     ) -> None:
 23         self.nickname = nickname
 24         self.rng = rng or random.Random()
 25         facts_path = facts_path or Path(__file__).with_name("data") / "facts.txt"
 26         try:
 27             self.facts = [
```

bot/commands.py (continued)

```
                line.strip()
                for line in facts_path.read_text(encoding="utf-8").splitlines()
                if line.strip() and "\x00" not in line
            ]
        except (OSError, UnicodeError):
            self.facts = []
        if not self.facts:
            self.facts = [
                "The network is made of packets, and packets travel one hop at a time."
            ]

    def channel_command(
        self,
        text: str,
        sender: str,
        channel_members: set[str],
        *,
        members_complete: bool = True,
    ) -> str | CommandAction | None:
        if not text.startswith("!"):
            return None
        command, _, argument = text[1:].partition(" ")
        command = command.casefold()
        if command in {"slap", "who"} and not members_complete:
            return CommandAction.WAIT_FOR_NAMES
        if command == "hello":
            return f"Hello, {sender}!"
        if command == "slap":
            return self._slap(argument.strip(), sender, channel_members)
        if command == "who":
            return CommandAction.REQUEST_WHO
        if command in {"channels", "network"}:
            if argument.strip():
                return f"Usage: !{command}"
            return (
                CommandAction.REQUEST_CHANNELS
                if command == "channels"
                else CommandAction.REQUEST_NETWORK
            )
        if command == "time":
            return "Bot time: " + datetime.now().astimezone().isoformat(
                sep=" ", timespec="seconds"
            )
        return None

    def _slap(self, target: str, sender: str, members: set[str]) -> str:
        folded_members = {irc_casefold(name): name for name in members}
        bot_key = irc_casefold(self.nickname)
        sender_key = irc_casefold(sender)
        eligible = {
            name
            for key, name in folded_members.items()
            if key not in {bot_key, sender_key}
        }
        # The no-argument form excludes both the bot and requester. An
        # explicit absent target falls back to the requester as required.
        target_key = irc_casefold(target)
        if target and target_key in {bot_key, sender_key}:
            return "Choose a channel member other than yourself or the Bot."
        if target and target_key in folded_members:
```

bot/commands.py (continued)

```
            chosen = folded_members[target_key]
        elif target:
            chosen = sender
        elif eligible:
            chosen = self.rng.choice(sorted(eligible))
        else:
            return "No eligible channel member to slap."
        return f"{sender} slaps {chosen} around a bit with a large trout!"

    def private_reply(self) -> str:
        return self.rng.choice(self.facts)
```

bot/data/facts.txt

```
Octopuses have three hearts.
The first email was sent in 1971.
IPv6 addresses are written in hexadecimal groups.
TCP delivers an ordered byte stream, not individual messages.
Honey can remain edible for a very long time when stored correctly.
```

bot/queries.py

```
"""IRC query state and response generation, independent of sockets and clocks."""

from __future__ import annotations

from dataclasses import dataclass, field

from common.irc_message import IRCMessage, message
from common.numerics import RPL_LIST, RPL_LISTEND, RPL_LUSERCLIENT
from common.protocol import irc_casefold, is_channel


@dataclass
class _WhoQuery:
    """One ISON batch group backing a channel's !who request."""

    channel: str
    started: float
    pending: int
    names: set[str] = field(default_factory=set)
    failed: bool = False


class BotQueries:
    WHO_TIMEOUT = 30.0
    QUERY_TIMEOUT = 30.0
    MAX_LISTED_CHANNELS = 10

    def __init__(self) -> None:
        self._who: _WhoQuery | None = None
        self._extra_queries: dict[str, tuple[str, float]] = {}
        self._listed_channels: list[tuple[str, str]] = []
        self._listed_count = 0

    def reset(self) -> None:
        """Discard all results and deadlines from a previous connection."""
        self._who = None
```

bot/queries.py (continued)

```
        self._reset_extra_queries()

    def _reset_extra_queries(self) -> None:
        self._extra_queries.clear()
        self._listed_channels.clear()
        self._listed_count = 0

    def start_who(
        self,
        channel: str,
        names: set[str],
        nickname: str,
        server_name: str,
        now: float,
    ) -> list[IRCMessage]:
        if self._who is not None:
            return []
        # miniircd's WHO reply has an unescaped IPv6 host parameter. ISON
        # confirms known members online without depending on that field.
        batches: list[list[str]] = []
        selected: list[str] = []
        for name in sorted(names):
            candidate = message("ISON", *(selected + [name]))
            reply = message(
                "303", nickname, " ".join(selected + [name]), prefix=server_name
            )
            # Budget the reply prefix as well as the request and parameter cap.
            if (
                len(selected) >= 14
                or len(candidate.to_bytes()) > 512
                or len(reply.to_bytes()) > 512
            ):
                batches.append(selected)
                selected = []
            selected.append(name)
        batches.append(selected or [nickname])
        self._who = _WhoQuery(channel=channel, started=now, pending=len(batches))
        return [message("ISON", *batch) for batch in batches]

    def start(self, command: str, channel: str, now: float) -> list[IRCMessage]:
        if command not in {"LIST", "LUSERS"}:
            raise ValueError("expected LIST or LUSERS")
        if command in self._extra_queries:
            return [
                message(
                    "PRIVMSG",
                    channel,
                    f"{command} query already in progress; please retry.",
                )
            ]
        self._extra_queries[command] = (channel, now)
        return [message(command)]

    def check_timeouts(self, now: float) -> None:
        if (
            self._who is not None
            and now - self._who.started >= self.WHO_TIMEOUT
        ):
            self._who = None
            # The caller reconnects so late replies cannot complete a new query.
```

bot/queries.py (continued)

```
            raise TimeoutError("ISON query timed out")
        for command, (_, started) in self._extra_queries.items():
            if now - started >= self.QUERY_TIMEOUT:
                self._reset_extra_queries()
                raise TimeoutError(f"{command} query timed out")

    def handle(self, incoming: IRCMessage, nickname: str) -> list[IRCMessage] | None:
        """Return outgoing messages, or None when this is not a query reply."""
        if (
            not incoming.params
            or irc_casefold(incoming.params[0]) != irc_casefold(nickname)
        ):
            return None
        outgoing = self._handle_extra_reply(incoming)
        if outgoing is not None:
            return outgoing
        who = self._who
        if who is None:
            return None
        if (
            incoming.command in {"263", "417", "421", "461"}
            and len(incoming.params) >= 2
            and incoming.params[1].upper() == "ISON"
        ):
            who.failed = True
            return self._finish_who_batch(who)
        if incoming.command == "303" and len(incoming.params) >= 2:
            who.names.update(incoming.params[-1].split())
            return self._finish_who_batch(who)
        return None

    def _finish_who_batch(self, who: _WhoQuery) -> list[IRCMessage]:
        who.pending -= 1
        if who.pending > 0:
            return []
        self._who = None
        if who.failed:
            return [
                message("PRIVMSG", who.channel, "WHO unavailable: server rejected ISON.")
            ]
        names = sorted(who.names)
        return [
            *(message("PRIVMSG", who.channel, f"WHO member: {name}") for name in names),
            message("PRIVMSG", who.channel, f"WHO complete: {len(names)} members"),
        ]

    def _handle_extra_reply(self, incoming: IRCMessage) -> list[IRCMessage] | None:
        params = incoming.params
        if incoming.command in {"263", "421", "461"} and len(params) >= 2:
            return self._reject_extra_query(params[1].upper())
        if incoming.command == RPL_LUSERCLIENT and len(params) >= 2:
            pending = self._extra_queries.pop("LUSERS", None)
            if pending is not None:
                return [message("PRIVMSG", pending[0], "NETWORK: " + params[-1])]
            return []
        return self._handle_list_reply(incoming)

    def _reject_extra_query(self, query: str) -> list[IRCMessage] | None:
        pending = self._extra_queries.pop(query, None)
        if pending is None:
```

bot/queries.py (continued)

```
157             return None
158         if query == "LIST":
159             self._listed_channels.clear()
160             self._listed_count = 0
161         return [
162             message(
163                 "PRIVMSG", pending[0], f"{query} unavailable: server rejected {query}."
164             )
165         ]
166 
167     def _handle_list_reply(self, incoming: IRCMessage) -> list[IRCMessage] | None:
168         params = incoming.params
169         pending = self._extra_queries.get("LIST")
170         if pending is None:
171             return None
172         if incoming.command == RPL_LIST and len(params) >= 4:
173             channel, count = params[1:3]
174             if (
175                 is_channel(channel)
176                 and len(count) <= 10
177                 and count.isascii()
178                 and count.isdigit()
179             ):
180                 self._listed_count += 1
181                 if len(self._listed_channels) < self.MAX_LISTED_CHANNELS:
182                     self._listed_channels.append((channel, count))
183             return []
184         if incoming.command == RPL_LISTEND and len(params) >= 2:
185             return self._finish_list(pending[0])
186         return None
187 
188     def _finish_list(self, destination: str) -> list[IRCMessage]:
189         del self._extra_queries["LIST"]
190         outgoing = [
191             message("PRIVMSG", destination, f"CHANNEL: {channel} ({count} users)")
192             for channel, count in self._listed_channels
193         ]
194         outgoing.append(
195             message(
196                 "PRIVMSG",
197                 destination,
198                 f"CHANNELS complete: {self._listed_count} channels "
199                 f"(showing {len(self._listed_channels)})",
200             )
201         )
202         self._listed_channels.clear()
203         self._listed_count = 0
204         return outgoing
```

bot/state.py

```python
"""Bot-side channel membership tracking."""

from __future__ import annotations

from common.protocol import irc_casefold


class BotState:
    def __init__(self) -> None:
        self.channels: dict[str, set[str]] = {}
        self._names_in_progress: dict[str, set[str]] = {}
        self._names_events: dict[str, list[tuple[str, tuple[str, ...]]]] = {}

    def joined(self, channel: str, nickname: str) -> None:
        self._record_event(channel, "joined", channel, nickname)
        # Keep IRC-equivalent keys stable while preserving nickname spelling.
        members = self.channels.setdefault(irc_casefold(channel), set())
        if self._matching_name(members, nickname) is None:
            members.add(nickname)

    @staticmethod
    def _matching_name(members: set[str], nickname: str) -> str | None:
        nickname_key = irc_casefold(nickname)
        return next((name for name in members if irc_casefold(name) == nickname_key), None)

    def left(self, channel: str, nickname: str) -> None:
        self._record_event(channel, "left", channel, nickname)
        members = self.channels.get(irc_casefold(channel))
        if members:
            existing = self._matching_name(members, nickname)
            if existing is not None:
                members.discard(existing)

    def renamed(self, old: str, new: str) -> None:
        for channel in self._names_in_progress:
            self._record_event(channel, "renamed", old, new)
        for members in self.channels.values():
            existing = self._matching_name(members, old)
            if existing is not None:
                members.discard(existing)
                members.add(new)

    def removed(self, nickname: str) -> None:
        for channel in self._names_in_progress:
            self._record_event(channel, "removed", nickname)
        for members in self.channels.values():
            existing = self._matching_name(members, nickname)
            if existing is not None:
                members.discard(existing)

    def members(self, channel: str) -> set[str]:
        return set(self.channels.get(irc_casefold(channel), set()))

    def clear(self) -> None:
        """Discard membership learned from a previous TCP connection."""
        self.channels.clear()
        self._names_in_progress.clear()
        self._names_events.clear()

    def _record_event(self, channel: str, event: str, *args: str) -> None:
```

bot/state.py (continued)

```
61          events = self._names_events.get(irc_casefold(channel))
62          if events is not None:
63              events.append((event, args))
64
65      def begin_names(self, channel: str) -> None:
66          self._names_in_progress[irc_casefold(channel)] = set()
67          self._names_events[irc_casefold(channel)] = []
68
69      def add_names(self, channel: str, names: list[str]) -> None:
70          if not self.has_pending_names(channel):
71              self.begin_names(channel)
72          pending = self._names_in_progress[irc_casefold(channel)]
73          for name in names:
74              if self._matching_name(pending, name) is None:
75                  pending.add(name)
76
77      def has_pending_names(self, channel: str) -> bool:
78          return irc_casefold(channel) in self._names_in_progress
79
80      def finish_names(self, channel: str) -> None:
81          channel_key = irc_casefold(channel)
82          if channel_key not in self._names_in_progress:
83              return
84          names = self._names_in_progress.pop(channel_key, set())
85          self.channels[channel_key] = names
86          # Apply events after the entire snapshot, including events preceding a
87          # later 353 chunk that still contains an old nickname.
88          events = self._names_events.pop(channel_key, [])
89          for event, args in events:
90              if event == "renamed":
91                  old, new = args
92                  existing = self._matching_name(names, old)
93                  if existing is not None:
94                      names.discard(existing)
95                      names.add(new)
96              elif event == "removed":
97                  existing = self._matching_name(names, args[0])
98                  if existing is not None:
99                      names.discard(existing)
100             else:
101                 getattr(self, event)(*args)
102
103     def replace_names(self, channel: str, names: list[str]) -> None:
104         # NAMES is authoritative for the snapshot; later events update this set.
105         self.begin_names(channel)
106         self.add_names(channel, names)
107         self.finish_names(channel)
```

common/__init__.py

```python
"""Shared IRC protocol primitives."""
```

common/irc_message.py

```python
"""RFC 2812 style IRC message parsing and serialization."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional


class IRCParseError(ValueError):
    """Raised when an IRC line cannot be represented as an IRC message."""


@dataclass(frozen=True)
class IRCMessage:
    """A single IRC message without its CRLF terminator."""

    command: str
    params: tuple[str, ...] = ()
    prefix: Optional[str] = None

    def __post_init__(self) -> None:
        command = self.command.strip()
        if (
            not command
            or not command.isascii()
            or not (command.isalpha() or (len(command) == 3 and command.isdigit()))
        ):
            raise IRCParseError("command must be a non-empty token")
        object.__setattr__(self, "command", command.upper())
        object.__setattr__(self, "params", tuple(self.params))
        if self.prefix is not None and (
            not self.prefix
            or any(ch.isspace() or ch in "\r\n\x00" for ch in self.prefix)
        ):
            raise IRCParseError("prefix must be a non-empty token")
        if len(self.params) > 15:
            raise IRCParseError("IRC messages have at most 15 parameters")
        for param in self.params:
            if any(char in param for char in "\r\n\x00"):
                raise IRCParseError("parameters cannot contain CR, LF or NUL")

    @classmethod
    def parse(cls, line: str) -> "IRCMessage":
        """Parse one IRC line, keeping a colon-prefixed trailing field intact."""
        if line.endswith("\r\n"):
            line = line[:-2]
        if any(char in line for char in "\r\n\x00"):
            raise IRCParseError("IRC line cannot contain embedded CR, LF or NUL")
        if not line:
            raise IRCParseError("empty IRC line")

        prefix = None
        if line.startswith(":"):
            try:
                prefix, line = line[1:].split(" ", 1)
```

common/irc_message.py (continued)

```
            except ValueError as exc:
                raise IRCParseError("prefix is missing a command") from exc
            if not prefix:
                raise IRCParseError("empty prefix")

        line = line.lstrip(" ")
        if not line:
            raise IRCParseError("command is missing")
        if " :" in line:
            head, trailing = line.split(" :", 1)
            parts = head.split()
            parts.append(trailing)
        else:
            parts = line.split()
        if not parts:
            raise IRCParseError("command is missing")
        return cls(parts[0], tuple(parts[1:]), prefix)

    def serialize(self) -> str:
        """Return the wire representation without CRLF.

        Only the final parameter can contain spaces, so it is emitted as the
        IRC trailing field when needed. Keeping CRLF in ``to_bytes`` makes the
        message boundary explicit at the socket layer.
        """
        pieces = []
        if self.prefix is not None:
            pieces.append(f":{self.prefix}")
        pieces.append(self.command)
        if self.params:
            for index, param in enumerate(self.params):
                is_last = index == len(self.params) - 1
                if not is_last and (
                    not param
                    or any(ch.isspace() for ch in param)
                    or param.startswith(":")
                ):
                    raise IRCParseError(
                        "only the final parameter may contain spaces or start with ':'"
                    )
                if is_last and (
                    not param
                    or any(ch.isspace() for ch in param)
                    or param.startswith(":")
                ):
                    # The leading colon here is the wire delimiter. Preserve a
                    # second colon when the logical value itself starts with one.
                    pieces.append(":" + param)
                else:
                    pieces.append(param)
        return " ".join(pieces)

    def to_bytes(self) -> bytes:
        return (self.serialize() + "\r\n").encode("utf-8")

    def to_bounded_bytes(
        self, max_line_bytes: int = 512, *, truncate_trailing: bool = False
    ) -> bytes:
        """Bound outgoing lines, optionally shortening a human-readable reason."""
        payload = self.to_bytes()
```

common/irc_message.py (continued)

```
116         if len(payload) <= max_line_bytes:
117             return payload
118         if not truncate_trailing or not self.params:
119             raise IRCParseError("outgoing IRC line exceeds maximum length")
120         empty = IRCMessage(self.command, (*self.params[:-1], ""), self.prefix)
121         available = max_line_bytes - len(empty.to_bytes())
122         if available < 0:
123             raise IRCParseError("IRC routing fields exceed maximum length")
124         # Cutting bytes can split a UTF-8 code point; discard only that tail.
125         trailing = (
126             self.params[-1].encode("utf-8")[:available].decode("utf-8", errors="ignore")
127         )
128         bounded = IRCMessage(self.command, (*self.params[:-1], trailing), self.prefix)
129         return bounded.to_bytes()
130
131
132 def message(command: str, *params: str, prefix: Optional[str] = None) -> IRCMessage:
133     return IRCMessage(command, tuple(params), prefix)
```

common/numerics.py

```
  1 """IRC numeric reply constants used by this implementation."""
  2
  3 RPL_WELCOME = "001"
  4 RPL_YOURHOST = "002"
  5 RPL_CREATED = "003"
  6 RPL_MYINFO = "004"
  7 RPL_LUSERCLIENT = "251"
  8 RPL_LIST = "322"
  9 RPL_LISTEND = "323"
 10 RPL_NAMREPLY = "353"
 11 RPL_ENDOFNAMES = "366"
 12 RPL_TIME = "391"
 13 RPL_WHOREPLY = "352"
 14 RPL_ENDOFWHO = "315"
 15
 16 ERR_NOSUCHNICK = "401"
 17 ERR_NOSUCHCHANNEL = "403"
 18 ERR_CANNOTSENDTOCHAN = "404"
 19 ERR_NONICKNAMEGIVEN = "431"
 20 ERR_ERRONEUSNICKNAME = "432"
 21 ERR_NICKNAMEINUSE = "433"
 22 ERR_NOTONCHANNEL = "442"
 23 ERR_NOTREGISTERED = "451"
 24 ERR_ALREADYREGISTERED = "462"
 25 ERR_UNKNOWNCOMMAND = "421"
 26 ERR_NEEDMOREPARAMS = "461"
 27 ERR_INPUTTOOLONG = "417"
```

common/protocol.py

```
"""Protocol-level names and validation shared by client and server."""

IRC_COMMANDS = frozenset(
    {"NICK", "USER", "JOIN", "PART", "PRIVMSG", "NAMES", "QUIT", "PING", "PONG", "TIME", "ISON", "LIST", "LUSERS"}
)
CHANNEL_PREFIXES = ("#", "&")


def irc_casefold(name: str) -> str:
    """Apply RFC 1459 casemapping to nicknames and channel names."""
    return name.translate(str.maketrans("ABCDEFGHIJKLMNOPQRSTUVWXYZ[]\\^", "abcdefghijklmnopqrstuvwxyz{}|~"))


def is_channel(target: str) -> bool:
    """Return whether *target* is a valid channel name in this IRC subset.

    This subset accepts 2-50 characters, starting with '#' or '&', and only
    printable ASCII excluding spaces, commas, and colons. The ASCII restriction
    is a local choice; the 50-character limit follows RFC 2812. Sharing this
    check keeps the server and bot consistent about valid targets.
    """
    if not target or target[0] not in CHANNEL_PREFIXES:
        return False
    if not 2 <= len(target) <= 50:
        return False
    return all(33 <= ord(char) <= 126 and char not in ",:" for char in target)


def valid_nickname(nickname: str) -> bool:
    """Apply the compact nickname grammar supported by this server."""
    if not 1 <= len(nickname) <= 30:
        return False
    ascii_letters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    special = "[]\\`_^{|}"
    if not (nickname[0] in ascii_letters or nickname[0] in special):
        return False
    return all(ch in ascii_letters + "0123456789" + special + "-" for ch in nickname)


def valid_username(username: str) -> bool:
    """Apply the local 1-30 ASCII character limit for user prefix fields."""
    return 1 <= len(username) <= 30 and all(
        33 <= ord(char) <= 126 and char not in "!@:" for char in username
    )
```

common/stream.py

```python
"""TCP stream framing for IRC's CRLF-delimited messages."""

from __future__ import annotations

from .irc_message import IRCMessage, IRCParseError


class IRCStreamError(ValueError):
    """Raised for malformed or over-sized IRC stream data."""


class IRCStreamDecoder:
    """Incrementally converts arbitrary TCP chunks into IRC messages."""

    def __init__(self, max_line_bytes: int = 512) -> None:
        self.max_line_bytes = max_line_bytes
        self._buffer = bytearray()

    def feed(self, data: bytes) -> list[IRCMessage]:
        if not isinstance(data, (bytes, bytearray, memoryview)):
            raise TypeError("data must be bytes-like")
        self._buffer.extend(data)
        messages: list[IRCMessage] = []
        while True:
            # TCP preserves byte order but not application write boundaries;
            # decode every complete CRLF line and retain any unfinished tail.
            delimiter = self._buffer.find(b"\r\n")
            if delimiter < 0:
                if len(self._buffer) > self.max_line_bytes:
                    self._buffer.clear()
                    raise IRCStreamError("IRC line exceeds maximum length")
                break
            raw = bytes(self._buffer[:delimiter])
            del self._buffer[: delimiter + 2]
            if len(raw) + 2 > self.max_line_bytes:
                raise IRCStreamError("IRC line exceeds maximum length")
            try:
                messages.append(IRCMessage.parse(raw.decode("utf-8")))
            except (UnicodeDecodeError, IRCParseError) as exc:
                raise IRCStreamError(str(exc)) from exc
        return messages
```

server/__init__.py

```
"""IPv6 IRC server implementation."""
```

server/__main__.py

```
from .cli import main

if __name__ == "__main__":
    main()
```

server/cli.py

```
"""Command-line configuration for the IRC server."""

import argparse
import logging

from .server import IRCServer


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="IPv6 IRC server")
    parser.add_argument("--host", default="::", help="IPv6 listen address")
    parser.add_argument("--port", type=int, default=6667)
    parser.add_argument("--name", default="irc.local", dest="server_name")
    return parser


def main(argv: list[str] | None = None) -> None:
    args = build_parser().parse_args(argv)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    try:
        server = IRCServer(args.host, args.port, args.server_name)
    except ValueError as exc:
        build_parser().error(str(exc))
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        server.stop()
```

server/client_session.py

```python
"""One TCP client session, isolated from protocol state and dispatch."""

from __future__ import annotations

import logging
from queue import Empty, Full, Queue
import socket
import threading
import time
from typing import TYPE_CHECKING

from common.irc_message import IRCMessage
from common.stream import IRCStreamDecoder, IRCStreamError

from .state import ClientState

if TYPE_CHECKING:
    from .server import IRCServer

logger = logging.getLogger(__name__)


class ClientSession:
    MAX_PENDING_MESSAGES = 256
    OUTPUT_QUEUE_TIMEOUT = 1.0

    def __init__(self, server: IRCServer, sock: socket.socket, address: tuple) -> None:
        self.server = server
        self.socket = sock
        self.address = address
        connected_at = server.monotonic()
        self.client = ClientState(
            self, address, last_activity=connected_at, connected_at=connected_at
        )
        self.decoder = IRCStreamDecoder()
        self.send_lock = threading.Condition()
        self.closed = threading.Event()
        self._outgoing: Queue[tuple[IRCMessage, bytes] | None] = Queue(
            self.MAX_PENDING_MESSAGES
        )
        self._writer: threading.Thread | None = None
        self._error_sent = False

    def start(self) -> None:
        with self.server.state.locked():
            # Shutdown snapshots clients under the same lock, so an accepted
            # socket cannot register after that snapshot and escape cleanup.
            if self.server.stopping.is_set():
                self.close()
                return
            self.server.state.add_client(self.client)
        try:
            threading.Thread(
                target=self.run, name=f"irc-client-{self.address}", daemon=True
            ).start()
        except BaseException:
            self.server.disconnect(
                self.client, "Session startup failed", announce=False
            )
            raise
```

server/client_session.py (continued)

```
61
62      def send(self, msg: IRCMessage) -> None:
63          self._enqueue(msg, wait_for_space=False)
64
65      def send_reply(self, msg: IRCMessage) -> None:
66          self.server.state.require_unlocked()
67          self._enqueue(msg, wait_for_space=True)
68
69      def wait_for_output(self) -> None:
70          """Pace request processing before it produces more notifications."""
71          self.server.state.require_unlocked()
72          deadline = time.monotonic() + self.OUTPUT_QUEUE_TIMEOUT
73          with self.send_lock:
74              while not self.closed.is_set():
75                  # Leave room for atomic notifications and multi-line replies.
76                  if self._outgoing.qsize() <= self.MAX_PENDING_MESSAGES // 2:
77                      return
78                  remaining = deadline - time.monotonic()
79                  if remaining <= 0:
80                      raise ConnectionError("client output queue did not drain")
81                  self.send_lock.wait(remaining)
82              raise ConnectionError("session is closed")
83
84      def _enqueue(self, msg: IRCMessage, *, wait_for_space: bool) -> None:
85          payload = msg.to_bounded_bytes(
86              truncate_trailing=msg.command in {"PART", "QUIT", "ERROR"}
87          )
88          deadline = time.monotonic() + self.OUTPUT_QUEUE_TIMEOUT
89          with self.send_lock:
90              while True:
91                  if self.closed.is_set():
92                      raise ConnectionError("session is closed")
93                  try:
94                      self._outgoing.put_nowait((msg, payload))
95                      return
96                  except Full as exc:
97                      if not wait_for_space:
98                          raise ConnectionError("client output queue is full") from exc
99                      remaining = deadline - time.monotonic()
100                     if remaining <= 0:
101                         raise ConnectionError("client output queue did not drain") from exc
102                     # Only a reply producer outside the state lock may wait.
103                     self.send_lock.wait(remaining)
104
105     def _write_loop(self) -> None:
106         try:
107             while not self.closed.is_set():
108                 item = self._outgoing.get()
109                 with self.send_lock:
110                     self.send_lock.notify_all()
111                 if item is None or self.closed.is_set():
112                     return
113                 msg, payload = item
114                 if msg.command == "PING":
115                     # Set before sendall so a fast PONG cannot beat this update.
116                     self.server.state.start_ping(self.client, self.server.monotonic())
117                 self.socket.sendall(payload)
118                 if msg.command == "ERROR":
119                     with self.send_lock:
120                         self._error_sent = True
```

server/client_session.py (continued)

```
                        self.send_lock.notify_all()
        except (ConnectionError, OSError) as exc:
            logger.info("IRC write failed for %s: %s", self.address, exc)
            self.server.disconnect(self.client, "Connection lost")
        except Exception:
            logger.exception("Unhandled IRC writer error from %s", self.address)
            self.server.disconnect(self.client, "Output failure")

    def run(self) -> None:
        try:
            self.socket.settimeout(1.0)
            self._writer = threading.Thread(
                target=self._write_loop, name=f"irc-writer-{self.address}", daemon=True
            )
            self._writer.start()
            while not self.closed.is_set() and not self.server.stopping.is_set():
                try:
                    # A timeout lets the thread notice server shutdown without
                    # using a busy loop or blocking forever on a dead peer.
                    data = self.socket.recv(4096)
                except socket.timeout:
                    continue
                if not data:
                    break
                try:
                    # One recv may contain half a message or many messages;
                    # framing is deliberately delegated to the shared decoder.
                    messages = self.decoder.feed(data)
                except IRCStreamError as exc:
                    logger.warning("Malformed IRC input from %s: %s", self.address, exc)
                    self.server.send_error_line(self.client, "Malformed IRC line")
                    with self.send_lock:
                        self.send_lock.wait_for(
                            lambda: self._error_sent or self.closed.is_set(),
                            timeout=self.OUTPUT_QUEUE_TIMEOUT,
                        )
                    break
                for incoming in messages:
                    if self.closed.is_set():
                        break
                    self.server.handlers.handle(self.client, incoming)
        except (ConnectionError, OSError) as exc:
            logger.info("IRC client disconnected: %s (%s)", self.address, exc)
        except Exception:
            # An unexpected bug must be visible during VM testing, while the
            # finally block still guarantees state and socket cleanup.
            logger.exception("Unhandled IRC session error from %s", self.address)
        finally:
            # All exits, including malformed input and TCP resets, pass through
            # the same cleanup path so channel membership cannot become stale.
            self.server.disconnect(self.client, "Connection closed", announce=True)
            if self._writer is not None and self._writer.ident is not None:
                self._writer.join(timeout=2)

    def close(self) -> None:
        with self.send_lock:
            if self.closed.is_set():
                return
            self.closed.set()
            # Discard pending output and wake an idle writer immediately.
```

server/client_session.py (continued)

```
            while True:
                try:
                    self._outgoing.get_nowait()
                except Empty:
                    break
            self._outgoing.put_nowait(None)
            self.send_lock.notify_all()
        try:
            self.socket.shutdown(socket.SHUT_RDWR)
        except OSError as exc:
            logger.debug(
                "Socket shutdown already complete for %s: %s", self.address, exc
            )
        try:
            self.socket.close()
        except OSError as exc:
            logger.debug("Socket close failed for %s: %s", self.address, exc)
```

server/errors.py

```
"""Domain exceptions raised by the server state and command layers."""


class NicknameInUseError(ValueError):
    """Raised when a client tries to claim an occupied IRC nickname."""
```

server/handlers.py

```
"""IRC command dispatch, validation and response routing."""

from __future__ import annotations

from datetime import datetime, timezone

from common import numerics
from common.irc_message import IRCMessage, message
from common.protocol import (
    CHANNEL_PREFIXES,
    IRC_COMMANDS,
    is_channel,
    valid_nickname,
    valid_username,
)

from .errors import NicknameInUseError
from .interfaces import HandlerServer
from .state import ClientState


class IRCHandlers:
    def __init__(self, server: HandlerServer) -> None:
        self.server = server

    def handle(self, client: ClientState, incoming: IRCMessage) -> None:
        if client.disconnected:
            raise ConnectionError("client has disconnected")
        client.last_activity = self.server.monotonic()
        command = incoming.command
```

server/handlers.py (continued)

```
        # Keep liveness acknowledgements and immediate cleanup unthrottled.
        if command not in {"PONG", "QUIT"} and not self.server.prepare_output(client):
            return
        if command not in IRC_COMMANDS:
            numeric = (
                numerics.ERR_NOTREGISTERED
                if not client.registered
                else numerics.ERR_UNKNOWNCOMMAND
            )
            self.error(
                client,
                numeric,
                command,
                "Unknown command" if client.registered else "You have not registered",
            )
            return
        if (
            command not in {"NICK", "USER", "QUIT", "PING", "PONG"}
            and not client.registered
        ):
            self.error(client, numerics.ERR_NOTREGISTERED, command)
            return
        # Keep each command in its own method so protocol parsing stays out of
        # socket code and individual IRC behaviors can be tested independently.
        handler = getattr(self, f"cmd_{command.lower()}", None)
        if handler is None:
            self.error(client, numerics.ERR_UNKNOWNCOMMAND, command, "Unknown command")
            return
        # Expected input errors are handled by each command and become IRC
        # numerics. Unexpected programming errors are allowed to reach the
        # session boundary, where they are logged with their traceback.
        handler(client, incoming.params)

    def error(self, client: ClientState, numeric: str, *params: str) -> None:
        target = client.nick or "*"
        # Invalid input must not become an invalid middle field in the error reply.
        params = tuple(
            "*"
            if index < len(params) - 1
            and (
                not value
                or len(value.encode("utf-8")) > 100
                or value.startswith(":")
                or any(char.isspace() for char in value)
            )
            else value
            for index, value in enumerate(params)
        )
        self.server.send(
            client, message(numeric, target, *params, prefix=self.server.server_name)
        )

    def cmd_nick(self, client: ClientState, params: tuple[str, ...]) -> None:
        if not params or not params[0]:
            self.error(client, numerics.ERR_NONICKNAMEGIVEN, "No nickname given")
            return
        nick = params[0]
        if not valid_nickname(nick):
            self.error(
                client, numerics.ERR_ERRONEUSNICKNAME, nick, "Erroneous nickname"
```

server/handlers.py (continued)

```
            )
            return
        try:
            old = self.server.set_nickname(client, nick)
        except NicknameInUseError:
            self.error(
                client, numerics.ERR_NICKNAMEINUSE, nick, "Nickname is already in use"
            )
            return
        if client.registered and old is None:
            self.welcome(client)

    def cmd_user(self, client: ClientState, params: tuple[str, ...]) -> None:
        if client.username is not None:
            self.error(client, numerics.ERR_ALREADYREGISTERED, "You may not reregister")
            return
        if len(params) < 4:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "USER", "Not enough parameters"
            )
            return
        if not valid_username(params[0]):
            self.error(
                client,
                numerics.ERR_NEEDMOREPARAMS,
                "USER",
                "Invalid username: use 1-30 printable ASCII characters excluding ! @ :",
            )
            return
        self.server.state.set_user(client, params[0], params[3])
        if client.registered:
            self.welcome(client)

    def welcome(self, client: ClientState) -> None:
        nick = client.nick or "*"
        prefix = self.server.server_name
        replies = (
            message(
                numerics.RPL_WELCOME,
                nick,
                f"Welcome to {self.server.server_name}",
                prefix=prefix,
            ),
            message(
                numerics.RPL_YOURHOST,
                nick,
                f"Your host is {self.server.server_name}",
                prefix=prefix,
            ),
            message(
                numerics.RPL_CREATED,
                nick,
                "IRC server ready",
                prefix=prefix,
            ),
            message(
                numerics.RPL_MYINFO,
                nick,
                self.server.server_name,
                "irc-mini",
```

server/handlers.py (continued)

```
                "io",
                "",
                prefix=prefix,
            ),
        )
        for reply in replies:
            self.server.send(client, reply)

    def cmd_join(self, client: ClientState, params: tuple[str, ...]) -> None:
        if not params or not params[0]:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "JOIN", "Not enough parameters"
            )
            return
        for channel in params[0].split(","):
            if not self.server.prepare_output(client):
                return
            if not is_channel(channel):
                self.error(
                    client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
                )
                continue
            if not self.server.join_channel(client, channel):
                continue
            # JOIN is echoed to the joining client as well as existing members;
            # NAMES then supplies the initial roster for clients joining late.
            if client.disconnected:
                return
            self.send_names(client, channel)

    def cmd_part(self, client: ClientState, params: tuple[str, ...]) -> None:
        if not params or not params[0]:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "PART", "Not enough parameters"
            )
            return
        reason = params[1] if len(params) > 1 else "Leaving"
        for channel in params[0].split(","):
            if not self.server.prepare_output(client):
                return
            if not is_channel(channel):
                self.error(
                    client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
                )
                continue
            if not self.server.part_channel(client, channel, reason):
                self.error(
                    client,
                    numerics.ERR_NOTONCHANNEL,
                    channel,
                    "You are not on that channel",
                )
                continue

    def cmd_names(self, client: ClientState, params: tuple[str, ...]) -> None:
        channels = (
            params[0].split(",")
            if params and params[0]
            else self.server.state.channels_for(client)
        )
```

server/handlers.py (continued)

```
        for channel in channels:
            if not self.server.prepare_output(client):
                return
            if not is_channel(channel):
                self.error(
                    client, numerics.ERR_NOSUCHCHANNEL, channel, "No such channel"
                )
                continue
            self.send_names(client, channel)

    def send_names(self, client: ClientState, channel: str) -> None:
        """Send the roster in 353 replies, then terminate it with a 366 reply.

        Split at nickname boundaries using the encoded reply size, including
        the server prefix, routing fields, and CRLF, against the 512-byte limit.
        """
        names = sorted(
            member.nick or "*" for member in self.server.state.channel_members(channel)
        )
        nick = client.nick or "*"
        prefix = self.server.server_name
        chunks: list[str] = []
        current: list[str] = []
        for name in names:
            candidate = " ".join((*current, name))
            reply = message(
                numerics.RPL_NAMREPLY, nick, "=", channel, candidate, prefix=prefix
            )
            if current and len(reply.to_bytes()) > 512:
                chunks.append(" ".join(current))
                current = [name]
            else:
                current.append(name)
        chunks.append(" ".join(current))
        for chunk in chunks:
            if client.disconnected:
                return
            self.server.send_reply(
                client,
                message(
                    numerics.RPL_NAMREPLY, nick, "=", channel, chunk, prefix=prefix
                ),
            )
        self.server.send_reply(
            client,
            message(
                numerics.RPL_ENDOFNAMES,
                nick,
                channel,
                "End of /NAMES list.",
                prefix=prefix,
            ),
        )

    def cmd_privmsg(self, client: ClientState, params: tuple[str, ...]) -> None:
        if len(params) < 2:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "PRIVMSG", "Not enough parameters"
            )
            return
```

server/handlers.py (continued)

```
271         target, text = params[0], params[1]
272         if not text:
273             self.error(client, "412", "No text to send")
274             return
275         outgoing = message("PRIVMSG", target, text, prefix=client.prefix)
276         if len(outgoing.to_bytes()) > 512:
277             self.error(client, numerics.ERR_INPUTTOOLONG, "PRIVMSG", "Message too long")
278             return
279         if target.startswith(CHANNEL_PREFIXES):
280             if not is_channel(target):
281                 self.error(
282                     client, numerics.ERR_NOSUCHCHANNEL, target, "No such channel"
283                 )
284                 return
285             members = self.server.state.channel_members(target)
286             if client not in members:
287                 self.error(
288                     client,
289                     numerics.ERR_CANNOTSENDTOCHAN,
290                     target,
291                     "Cannot send to channel",
292                 )
293                 return
294             # IRC channel messages are delivered to peers, not echoed back to
295             # the sender; the client already knows what it submitted.
296             self.server.send_many(
297                 (member for member in members if member is not client), outgoing
298             )
299             return
300         destination = self.server.state.direct_target(target)
301         if destination is None:
302             self.error(client, numerics.ERR_NOSUCHNICK, target, "No such nick")
303             return
304         self.server.send(destination, outgoing)
305 
306     def cmd_ping(self, client: ClientState, params: tuple[str, ...]) -> None:
307         token = params[0] if params else self.server.server_name
308         reply = message(
309             "PONG", self.server.server_name, token, prefix=self.server.server_name
310         )
311         if len(reply.to_bytes()) > 512:
312             # Optional routing fields must not displace the echoed PING token.
313             reply = message("PONG", token)
314         self.server.send(client, reply)
315 
316     def cmd_pong(self, client: ClientState, params: tuple[str, ...]) -> None:
317         if params and params[-1] == self.server.server_name:
318             self.server.state.acknowledge_ping(client)
319 
320     def cmd_time(self, client: ClientState, params: tuple[str, ...]) -> None:
321         now = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")
322         self.server.send(
323             client,
324             message(
325                 numerics.RPL_TIME,
326                 client.nick or "*",
327                 self.server.server_name,
328                 now,
329                 prefix=self.server.server_name,
330             ),
```

server/handlers.py (continued)

```
        )

    def cmd_ison(self, client: ClientState, params: tuple[str, ...]) -> None:
        if not params:
            self.error(
                client, numerics.ERR_NEEDMOREPARAMS, "ISON", "Not enough parameters"
            )
            return
        names: list[str] = []
        seen: set[str] = set()
        nick = client.nick or "*"
        prefix = self.server.server_name
        for name in (name for param in params for name in param.split()):
            target = self.server.state.direct_target(name)
            if target is None or not target.nick or target.nick in seen:
                continue
            names.append(target.nick)
            seen.add(target.nick)
        reply = message("303", nick, " ".join(names), prefix=prefix)
        if len(reply.to_bytes()) > 512:
            self.error(
                client,
                numerics.ERR_INPUTTOOLONG,
                "ISON",
                "Reply too long; use smaller batches",
            )
            return
        self.server.send(client, reply)

    def cmd_list(self, client: ClientState, params: tuple[str, ...]) -> None:
        requested = params[0].split(",") if params else None
        for name, count in self.server.state.list_channels(requested):
            if client.disconnected:
                return
            self.server.send_reply(
                client,
                message(
                    numerics.RPL_LIST, client.nick or "*", name, str(count), "",
                    prefix=self.server.server_name,
                ),
            )
        self.server.send_reply(
            client,
            message(
                numerics.RPL_LISTEND, client.nick or "*", "End of /LIST",
                prefix=self.server.server_name,
            ),
        )

    def cmd_lusers(self, client: ClientState, params: tuple[str, ...]) -> None:
        count = self.server.state.registered_user_count()
        self.server.send(
            client,
            message(
                numerics.RPL_LUSERCLIENT,
                client.nick or "*",
                f"There are {count} users and 0 services on 1 server",
                prefix=self.server.server_name,
            ),
        )
```

server/handlers.py (continued)

```
391
392     def cmd_quit(self, client: ClientState, params: tuple[str, ...]) -> None:
393         reason = params[0] if params else "Client Quit"
394         self.server.disconnect(client, reason)
```

server/interfaces.py

```
  1  """Structural contracts for command handling and client delivery."""
  2
  3  from __future__ import annotations
  4
  5  from collections.abc import Iterable
  6  from typing import TYPE_CHECKING, Protocol
  7
  8  from common.irc_message import IRCMessage
  9
 10  if TYPE_CHECKING:
 11      from .state import ClientState, IRCState
 12
 13
 14  class ClientConnection(Protocol):
 15      """Separate immediate enqueueing from bounded reply production."""
 16
 17      def send(self, msg: IRCMessage) -> None: ...
 18
 19      def send_reply(self, msg: IRCMessage) -> None: ...
 20
 21      def wait_for_output(self) -> None: ...
 22
 23      def close(self) -> None: ...
 24
 25
 26  class HandlerServer(Protocol):
 27      state: IRCState
 28      server_name: str
 29
 30      def monotonic(self) -> float: ...
 31
 32      def send(self, client: ClientState, msg: IRCMessage) -> None: ...
 33
 34      def send_reply(self, client: ClientState, msg: IRCMessage) -> None: ...
 35
 36      def prepare_output(self, client: ClientState) -> bool: ...
 37
 38      def send_many(self, clients: Iterable[ClientState], msg: IRCMessage) -> None: ...
 39
 40      def join_channel(self, client: ClientState, channel: str) -> bool: ...
 41
 42      def part_channel(self, client: ClientState, channel: str, reason: str) -> bool: ...
 43
 44      def set_nickname(self, client: ClientState, nick: str) -> str | None: ...
 45
 46      def disconnect(
 47          self, client: ClientState, reason: str = "Client Quit", announce: bool = True
 48      ) -> None: ...
```

server/server.py

```python
"""Small, concurrent IPv6 IRC server for a practical protocol subset."""

from __future__ import annotations

import logging
from collections import deque
import socket
import threading
import time
from collections.abc import Iterable

from common.irc_message import IRCMessage, message

from .client_session import ClientSession
from .handlers import IRCHandlers
from .state import ClientState, IRCState

logger = logging.getLogger(__name__)


class IRCServer:
    REGISTRATION_TIMEOUT = 30.0
    IDLE_TIMEOUT = 30.0
    PONG_TIMEOUT = 30.0

    def __init__(
        self, host: str = "::", port: int = 6667, server_name: str = "irc.local"
    ) -> None:
        if not 0 <= port <= 65535:
            raise ValueError("port must be between 0 and 65535")
        if (
            not 1 <= len(server_name) <= 63
            or not server_name.isascii()
            or any(not 33 <= ord(char) <= 126 or char == ":" for char in server_name)
        ):
            raise ValueError(
                "server_name must be 1-63 printable ASCII characters without spaces or ':'"
            )
        self.host = host
        self.port = port
        self.server_name = server_name
        self.state = IRCState()
        self.handlers = IRCHandlers(self)
        self.socket: socket.socket | None = None
        self.ready = threading.Event()
        self.stopping = threading.Event()
        self._monitor: threading.Thread | None = None

    @staticmethod
    def monotonic() -> float:
        return time.monotonic()

    def serve_forever(self) -> None:
        # AF_INET6 is explicit so this server accepts IPv6 clients
        # and bind to the unspecified address by default.
        try:
            # Publish the listener and readiness atomically with shutdown.
            with self.state.locked():
                if self.stopping.is_set():
                    return
```

server/server.py (continued)

```
                self.socket = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
                self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
                self.socket.bind((self.host, self.port))
                self.socket.listen(100)
                self.socket.settimeout(1.0)
                self.ready.set()
                self._monitor = threading.Thread(
                    target=self._monitor_clients, name="irc-monitor", daemon=True
                )
                self._monitor.start()
            while not self.stopping.is_set():
                try:
                    sock, address = self.socket.accept()
                except socket.timeout:
                    continue
                except OSError:
                    if self.stopping.is_set():
                        break
                    raise
                ClientSession(self, sock, address).start()
        finally:
            self.stop()

    def send(self, client: ClientState, msg: IRCMessage) -> None:
        self.send_many((client,), msg)

    def prepare_output(self, client: ClientState) -> bool:
        """Wait for request output headroom before entering a state transaction."""
        self.state.require_unlocked()
        if client.disconnected:
            return False
        try:
            client.connection.wait_for_output()
        except (ConnectionError, OSError) as exc:
            logger.info("Output stalled for %s: %s", client.address, exc)
            self.disconnect(client, "Output timeout")
            return False
        return True

    def send_reply(self, client: ClientState, msg: IRCMessage) -> None:
        """Allow bounded queue waits only outside a state transaction."""
        self.state.require_unlocked()
        if client.disconnected:
            return
        try:
            client.connection.send_reply(msg)
        except (ConnectionError, OSError) as exc:
            logger.info("Unable to send to %s: %s", client.address, exc)
            self.disconnect(client, "Connection lost")

    def send_many(self, clients: Iterable[ClientState], msg: IRCMessage) -> None:
        # Connection.send only enqueues; socket writes belong to each session.
        pending = deque([(list(clients), msg)])
        with self.state.locked():
            while pending:
                recipients, outgoing = pending.popleft()
                for client in recipients:
                    if client.disconnected:
                        continue
                    try:
```

server/server.py (continued)

```
                        client.connection.send(outgoing)
                    except (ConnectionError, OSError) as exc:
                        logger.info("Unable to send to %s: %s", client.address, exc)
                        peers = self._close_client(client)
                        if client.nick and peers:
                            pending.append(
                                (
                                    list(peers),
                                    message("QUIT", "Connection lost", prefix=client.prefix),
                                )
                            )

    def join_channel(self, client: ClientState, channel: str) -> bool:
        with self.state.locked():
            joined, members = self.state.join(client, channel)
            if joined:
                self.send_many(members, message("JOIN", channel, prefix=client.prefix))
        return joined

    def set_nickname(self, client: ClientState, nick: str) -> str | None:
        with self.state.locked():
            old_prefix = client.prefix
            old = self.state.set_nick(client, nick)
            if old and old != nick:
                recipients = {client}
                for channel in self.state.channels_for(client):
                    recipients.update(self.state.channel_members(channel))
                self.send_many(recipients, message("NICK", nick, prefix=old_prefix))
            return old

    def part_channel(self, client: ClientState, channel: str, reason: str) -> bool:
        with self.state.locked():
            parted, members = self.state.part(client, channel)
            if parted:
                self.send_many(
                    members, message("PART", channel, reason, prefix=client.prefix)
                )
            return parted

    def send_error_line(self, client: ClientState, detail: str) -> None:
        self.send(client, message("ERROR", detail, prefix=self.server_name))

    def disconnect(
        self, client: ClientState, reason: str = "Client Quit", announce: bool = True
    ) -> None:
        with self.state.locked():
            recipients = self._close_client(client)
            if announce and client.nick and recipients:
                self.send_many(recipients, message("QUIT", reason, prefix=client.prefix))

    def _close_client(self, client: ClientState) -> set[ClientState]:
        # Remove routes and close before any potentially blocking peer writes.
        affected = self.state.remove(client)
        client.connection.close()
        return {
            recipient
            for _channel, channel_recipients in affected
            for recipient in channel_recipients
        }

```

server/server.py (continued)

```
181     def _monitor_clients(self) -> None:
182         while not self.stopping.wait(1):
183             self._check_idle_clients()
184 
185     def _check_idle_clients(self) -> None:
186         now = self.monotonic()
187         with self.state.locked():
188             clients = list(self.state.clients)
189         for client in clients:
190             with self.state.locked():
191                 if client.disconnected:
192                     continue
193                 if (
194                     not client.registered
195                     and now - client.connected_at >= self.REGISTRATION_TIMEOUT
196                 ):
197                     # Keep expiration and removal atomic with NICK/USER updates.
198                     self.disconnect(client, "Registration timeout", announce=False)
199                     continue
200                 send_probe = (
201                     client.ping_sent is None
202                     and not client.ping_queued
203                     and now - client.last_activity >= self.IDLE_TIMEOUT
204                 )
205                 if send_probe:
206                     client.ping_queued = True
207                 timed_out = (
208                     client.ping_sent is not None
209                     and now - client.ping_sent >= self.PONG_TIMEOUT
210                 )
211             if send_probe:
212                 self.send(
213                     client, message("PING", self.server_name, prefix=self.server_name)
214                 )
215             if timed_out:
216                 self.disconnect(client, "Ping timeout")
217 
218     def stop(self) -> None:
219         with self.state.locked():
220             if self.stopping.is_set():
221                 return
222             self.stopping.set()
223             self.ready.clear()
224             clients = list(self.state.clients)
225         if self.socket:
226             try:
227                 self.socket.close()
228             except OSError as exc:
229                 logger.debug("Server listener close failed: %s", exc)
230         for client in clients:
231             self.disconnect(client, "Server shutting down", announce=False)
```

server/state.py

```python
"""Thread-safe in-memory IRC users, channels and routing state."""

from __future__ import annotations

import threading
import time
from collections.abc import Iterator
from contextlib import contextmanager
from dataclasses import dataclass, field

from .errors import NicknameInUseError
from .interfaces import ClientConnection
from common.protocol import irc_casefold


@dataclass(eq=False)
class ClientState:
    connection: ClientConnection
    address: tuple
    nick: str | None = None
    username: str | None = None
    realname: str | None = None
    channels: set[str] = field(default_factory=set)
    last_activity: float = field(default_factory=time.monotonic)
    ping_sent: float | None = None
    disconnected: bool = False
    connected_at: float = field(default_factory=time.monotonic)
    ping_queued: bool = False

    @property
    def registered(self) -> bool:
        return bool(self.nick and self.username)

    @property
    def prefix(self) -> str:
        nick = self.nick or "*"
        user = self.username or "unknown"
        host = self.address[0] if self.address else "unknown"
        return f"{nick}!{user}@{host}"


class IRCState:
    """Owns all mutable server state; callers must not retain internal sets."""

    def __init__(self) -> None:
        self._lock = threading.RLock()
        self._lock_owner: int | None = None
        self._lock_depth = 0
        self.clients: set[ClientState] = set()
        self.nicknames: dict[str, ClientState] = {}
        self.channels: dict[str, set[ClientState]] = {}

    @contextmanager
    def locked(self) -> Iterator[None]:
        """Keep state changes atomic and track nested ownership for delivery."""
        with self._lock:
            self._lock_owner = threading.get_ident()
            self._lock_depth += 1
            try:
                yield
```

server/state.py (continued)

```
            finally:
                self._lock_depth -= 1
                if self._lock_depth == 0:
                    self._lock_owner = None

    def require_unlocked(self) -> None:
        if self._lock_owner == threading.get_ident():
            raise RuntimeError("reply delivery cannot wait while holding IRC state")

    def add_client(self, client: ClientState) -> None:
        with self.locked():
            self._require_connected(client)
            self.clients.add(client)

    @staticmethod
    def _channel_key(channel: str) -> str:
        """IRC channel names are case-insensitive for routing and membership."""
        return irc_casefold(channel)

    def find_nick(self, nick: str) -> ClientState | None:
        with self.locked():
            return self.nicknames.get(irc_casefold(nick))

    def set_nick(self, client: ClientState, nick: str) -> str | None:
        with self.locked():
            self._require_connected(client)
            # IRC nickname comparisons are case-insensitive, while preserving
            # the spelling the client chose for display and wire replies.
            old = client.nick
            if (
                irc_casefold(nick) in self.nicknames
                and self.nicknames[irc_casefold(nick)] is not client
            ):
                raise NicknameInUseError("nickname in use")
            if old:
                self.nicknames.pop(irc_casefold(old), None)
            client.nick = nick
            self.nicknames[irc_casefold(nick)] = client
            return old

    def set_user(self, client: ClientState, username: str, realname: str) -> None:
        with self.locked():
            self._require_connected(client)
            client.username = username
            client.realname = realname

    def join(self, client: ClientState, channel: str) -> tuple[bool, list[ClientState]]:
        with self.locked():
            self._require_connected(client)
            # Return a snapshot so the caller can release the state lock before
            # performing potentially blocking socket writes.
            channel_key = self._channel_key(channel)
            members = self.channels.setdefault(channel_key, set())
            if client in members:
                return False, list(members)
            members.add(client)
            client.channels.add(channel_key)
            return True, list(members)

    @staticmethod
```

server/state.py (continued)

```
121     def _require_connected(client: ClientState) -> None:
122         if client.disconnected:
123             raise ConnectionError("client has disconnected")
124 
125     def part(self, client: ClientState, channel: str) -> tuple[bool, list[ClientState]]:
126         with self.locked():
127             channel_key = self._channel_key(channel)
128             members = self.channels.get(channel_key)
129             if not members or client not in members:
130                 return False, []
131             members.remove(client)
132             client.channels.discard(channel_key)
133             recipients = list(members) + [client]
134             if not members:
135                 self.channels.pop(channel_key, None)
136             return True, recipients
137 
138     def channel_members(self, channel: str) -> list[ClientState]:
139         with self.locked():
140             return list(self.channels.get(self._channel_key(channel), set()))
141 
142     def channels_for(self, client: ClientState) -> list[str]:
143         with self.locked():
144             return list(client.channels)
145 
146     def list_channels(self, requested: list[str] | None = None) -> list[tuple[str, int]]:
147         """Return channel counts without exposing live membership sets."""
148         keys = {irc_casefold(name) for name in requested} if requested is not None else None
149         with self.locked():
150             return sorted(
151                 (name, len(members))
152                 for name, members in self.channels.items()
153                 if keys is None or name in keys
154             )
155 
156     def registered_user_count(self) -> int:
157         with self.locked():
158             return sum(client.registered for client in self.clients)
159 
160     def acknowledge_ping(self, client: ClientState) -> None:
161         with self.locked():
162             client.ping_sent = None
163 
164     def start_ping(self, client: ClientState, now: float) -> None:
165         """Start the PONG deadline when the writer takes the queued probe."""
166         with self.locked():
167             if not client.disconnected and client.ping_queued:
168                 client.ping_queued = False
169                 client.ping_sent = now
170 
171     def direct_target(self, nick: str) -> ClientState | None:
172         with self.locked():
173             client = self.nicknames.get(irc_casefold(nick))
174             if client is not None and client in self.clients and client.registered:
175                 return client
176             return None
177 
178     def remove(self, client: ClientState) -> list[tuple[str, list[ClientState]]]:
179         """Remove a client and return channels plus remaining recipients."""
180         with self.locked():
```

server/state.py (continued)

```
            # Clean every index in one critical section so no later command can
            # route a message to a disconnected session.
            client.disconnected = True
            affected: list[tuple[str, list[ClientState]]] = []
            for channel in list(client.channels):
                members = self.channels.get(channel, set())
                members.discard(client)
                affected.append((channel, list(members)))
                if not members:
                    self.channels.pop(channel, None)
            client.channels.clear()
            self.clients.discard(client)
            if client.nick and self.nicknames.get(irc_casefold(client.nick)) is client:
                self.nicknames.pop(irc_casefold(client.nick), None)
            return affected
```